

(11) [20] PostgreSQL 벡터 데이터 관리의 근본적 한계

미팅 준비 심층분석

작성일: 2026-03-19 **대상:** Exqutor 졸업 논문 프로젝트 **원본 논문:** "Are There Fundamental Limitations in Supporting Vector Data Management in Relational Databases? A Case Study of PostgreSQL" (ICDE 2024, Purdue University)

1. 논문 총정리

1.1 핵심 질문과 대답

이 논문의 제목이 모든 것을 담고 있다:

"관계형 데이터베이스에서 벡터 데이터를 관리하는 데 **근본적인** 한계가 있는가?"

가능한 대답:

1. **Yes, 근본적 한계가 있다** — 관계형 DB와 벡터 DB는 설계 철학 자체가 다르다
2. **No, 단순 구현 문제다** — 현재 구현이 미흡할 뿐, 개선으로 충분하다
3. **Yes and No, 둘 다다** — 일부는 근본적 제약, 일부는 개선 가능

이 논문의 대답: 3번. "일부는 근본적 한계, 일부는 구현 개선 가능"

1.2 발견된 5가지 핵심 한계

논문을 체계적인 실험을 통해 pgvector가 Faiss(전문 벡터 DB)보다 성능이 떨어지는 이유를 5가지로 분류했다:

#	한계	원인	성능 영향	근본성
1	버퍼 풀 오버헤드	래치 경합	15~20배	근본적
2	튜플 접근 오버헤드	MVCC 메타데이터	20~50%	근본적
3	공간 증폭	8KB 페이지 구조	8배	근본적
4	인덱스 구축 시간	WAL 오버헤드	96배	부분 근본
5	고정 선택도	통계 불가능	425배	부분 근본

1.3 각 한계의 상세 설명

한계 1: 버퍼 풀(Buffer Pool) 오버헤드 — 래치 경합

PostgreSQL의 메모리 아키텍처:

PostgreSQL은 모든 데이터를 "페이지"(8KB 단위)로 관리한다. 데이터에 접근하려면:

```

프로세스 요청
↓
버퍼 풀 관리자 호출
↓
페이지 ID → 메모리 주소 해시 테이블 조회
↓
페이지 래치(Latch) 획득 (spin lock, 다른 프로세스 접근 방지)
↓
페이지 데이터 읽기
↓
래치 해제

```

HNSW 벡터 인덱스의 접근 패턴:

```

HNSW는 그래프 기반 검색:
현재 노드에서 이웃들의 거리 계산
→ 가장 가까운 이웃 찾기
→ 그 이웃으로 이동
→ 반복 (수십~수백 번의 노드 방문)

```

이는 무작위 접근(random access) 패턴이다. 즉, 매우 빈번하게 다양한 페이지에 접근해야 한다.

문제의 핵심: 래치 경합(Latch Contention)

HNSW 벡터 검색 100만 벡터에서 top-10 찾기:

초당 페이지 접근 횟수: 수만~수십만 회
각 접근마다: 래치 획득 → 데이터 읽기 → 래치 해제

래치 획득 시간 (spin lock):

경합 없음: ~50ns

경합 있음: ~500ns~1000ns

전체 검색 시간 분석:

거리 계산: 50ms (CPU 작업)

래치 오버헤드: 100ms (대기 시간)

비율: 66% 정도가 래치 때문

실제 성능 수치:

데이터셋: 100만 벡터 (1536차원, OpenAI 임베딩)

쿼리: top-10 검색

PostgreSQL pgvector: 150ms

Faiss (메모리 기반): 8ms

비율: $150/8 = 18.75$ 배 느림

대부분이 래치 오버헤드

왜 근본적 한계인가?

- 래치는 PostgreSQL의 동시성 제어 핵심
- 트랜잭션 안전성, 다중 사용자 지원을 위해 필수
- 이를 제거하면 PostgreSQL이 아님
- 벡터 DB는 이런 기능이 필요 없으므로 버퍼 풀을 생략 가능

개선 가능성: 제한적. VBASE(벡터 전용 버퍼 풀) 같은 아키텍처 변경 필요

한계 2: 튜플 접근(Tuple Access) 오버헤드 — MVCC

PostgreSQL의 튜플 저장 구조:

8KB 페이지:

페이지 헤더 (24B)
튜플 1:
├ 튜플 헤더 (23B)
│ ├ xmin (4B): 삽입 TXN ID
│ ├ xmax (4B): 삭제 TXN ID
│ ├ cmin/cmax (2B): CMD ID
│ ├ 플래그 (1B): 상태
│ └ 락 정보 (6B)
└ 데이터 (1KB 벡터)
튜플 2, 3, ...
여유 공간 (패딩)

벡터 검색 시 데이터 접근 과정:

1단계: 페이지 로드 (위의 버퍼 풀 과정)

2단계: 튜플 헤더 파싱

- xmin, xmax, cmin, cmax 읽기
- 현재 트랜잭션이 이 튜플을 볼 수 있는가? 확인

3단계: MVCC 가시성 검사

- 현재 스냅샷: TXN 1000이 진행 중
- 이 튜플은 TXN 800이 삽입, TXN 900이 삭제했음
- TXN 1000은 이 튜플을 볼 수 있나? YES/NO 판단
- 메모리 참조 5~10회 추가

4단계: 실제 벡터 데이터 추출

- 컬럼 오프셋 계산
- 벡터 데이터 메모리 접근 (1KB)

성능 영향:

벡터 1개 검색 비용 분석:

거리 계산: $1536 \text{ float} \times 4\text{B} = 6.144\text{KB}$ 메모리 접근

MVCC 체크: 튜플 헤더 파싱(200B) + 메모리 참조(10회 $\times$ 8B = 80B)

총 추가 오버헤드: 280B (=약 4% 추가)

100만 벡터 검색:

거리 계산만: 약 100ms

+ MVCC 오버헤드: 약 20~30ms

총: 120~130ms

왜 근본적 한계인가?

- MVCC는 PostgreSQL의 핵심 기능 (동시 읽기/쓰기 지원)
- 모든 튜플에 메타데이터가 필수
- 벡터 DB는 보통 읽기 전용이므로 이 기능이 불필요

개선 가능성: 중간 수준. 벡터를 특별히 취급하여 MVCC 체크 단순화 가능

한계 3: 공간 증폭(Space Amplification) — 8KB 페이지 구조

PostgreSQL 페이지의 구조적 낭비:

HNSW는 그래프 구조:

각 노드: 벡터 1536차원 (6KB)

간선: 이웃 포인터 약 40~50개 (200B)

디스크에 저장하려면 PostgreSQL 페이지에 들어가야 함:

페이지 1에 간선 정보 200B 저장

→ 나머지 7,800B는 패딩

→ 공간 낭비: 97.5%

이런 낭비가 누적되면:

원본 간선 정보: 100MB

PostgreSQL 저장: 약 800MB (8배 증폭)

실제 측정 결과:

데이터셋: 1,000만 벡터 (1536D)

원본 벡터 데이터: 60GB

Faiss HNSW 인덱스:

크기: ~90GB

메모리에 들어감 (모두 또는 대부분)

접근시간: ~100ns (메인 메모리)

PostgreSQL HNSW 인덱스:

크기: ~480GB (5.3배!)

메모리에 못 들어감 (상당 부분 SSD로 스푼)

접근시간: ~10,000ns (SSD) = 100배 느림

원인 분석:

PostgreSQL의 공간 증폭 요소:

1. 페이지 단위 저장
 - 8KB 페이지는 불변 단위
 - 가변 크기 데이터는 낭비 증가
2. MVCC 메타데이터
 - 각 벡터마다 xmin/xmax/cmin/cmax = 24B
 - 1000만 벡터 × 24B = 240MB 추가
3. 정렬 및 임시 공간
 - 인덱스 구축 중 정렬용 임시 공간
 - maintenance_work_mem 초과 시 디스크 사용
4. 튜플 헤더
 - 각 튜플마다 헤더 23B
 - 1000만 벡터 × 23B = 230MB 추가

왜 근본적 한계인가?

- 8KB 페이지는 PostgreSQL의 근본 설계 (동시성, 복구, WAL과 통합)
- 페이지 크기를 줄이면 I/O 횟수 증가
- 페이지 크기를 키우면 내부 단편화 증가
- 벡터 DB는 가변 크기 저장 가능

개선 가능성: 낮음. 아키텍처 수준의 변경 필요

한계 4: 인덱스 구축 시간 — WAL 오버헤드

PostgreSQL의 인덱스 구축 과정:

각 벡터를 삽입할 때:

1단계: 버퍼 풀에 벡터 저장

└ 래치 획득 → 데이터 기록 → 래치 해제

2단계: WAL(Write-Ahead Log)에 기록

└ "벡터 삽입" 로그 쓰기 (디스크 동기 I/O)

3단계: HNSW 그래프 갱신

└ 기존 노드들과 거리 계산

└ 가장 가까운 K개 이웃 선택

└ 그래프 연결 추가

└ 다시 버퍼 풀에 기록 + WAL 기록

4단계: maintenance_work_mem 부족 시

└ 스푼(메모리 → SSD) → 나중에 다시 읽기

결과: 벡터 1개 = 약 20~50회 페이지 I/O

성능 수치:

데이터셋: 1000만 벡터 (1536D, 약 60GB)

PostgreSQL pgvector:

구축 시간: 48시간

처리 속도: 약 57 벡터/초

Faiss:

구축 시간: 30분

처리 속도: 약 5,500 벡터/초

비율: $5,500 / 57 \approx 96$ 배 빠름

원인:

Faiss: 메모리에서 직접 작업 (I/O 거의 없음)

pgvector: 벡터 1개 = 20회 I/O (WAL 포함)

→ $1000만 \times 20 = 2억$ 회 페이지 I/O

→ 약 48시간 소요

WAL이 필수인 이유:

장애 복구 시나리오:

시점 A: 벡터 0~500만 개 저장 완료 (메모리)

시점 B: 500만 개 저장 중

└ 데이터베이스 서버 크래시

복구:

- WAL 로그 읽기: "500만 개 저장 시작"
- 이 시점에서 메모리는 날아감
- WAL에 완전히 기록된 벡터만 복구 가능
- 부분 기록된 벡터는 롤백

왜 부분적 근본 한계인가?

- WAL 자체는 필수 (장애 복구)
- 하지만 **인덱스 구축 시**에는 WAL을 우회 가능 (인덱스 재생성으로 해결)
- 또는 **병렬 구축** (여러 프로세스가 다른 파티션 처리)으로 개선 가능

개선 가능성: 높음. 구현 최적화로 상당히 개선 가능

한계 5: 고정 선택도(Fixed Selectivity) — 쿼리 최적화 한계

PostgreSQL의 카디널리티 추정:

일반적인 WHERE 조건에서는 통계를 사용한다:

```
WHERE age > 30
  → 히스토그램 사용
  → 정확도: ~90%

WHERE name = 'John'
  → 빈도 분석
  → 정확도: ~95%
```

하지만 벡터는 다르다:

```
WHERE embedding <-> query_vec < 0.5
```

pgvector는 이 거리 분포를 모름:

- "알 수 없음" 상태
- 기본값 사용: 선택도 = 33.3%

의미: 모든 벡터 쿼리는 데이터의 1/3을 반환한다고 가정

실제 데이터와의 괴리:

벤치마크: Wikipedia 이미지 임베딩 (1000만 개)

쿼리: 벡터 유사도 < 1.0 (가까운 100개 찾기)

실제 선택도: 0.001% (100 / 10,000,000)

pgvector 추정: 33.3%

오류: 33,300배 과대 추정!

또 다른 쿼리: 벡터 유사도 < 0.1 (가까운 1개 찾기)

실제 선택도: 0.00001%

pgvector 추정: 33.3%

오류: 3,330,000배 과대 추정!

쿼리 계획에 미치는 영향:

쿼리:

```
SELECT o.order_id, l.product_id, l.price
FROM orders o
JOIN lineitem l ON o.id = l.order_id
WHERE l.product_embedding <-> query_vec < 0.5
```

테이블 크기:

orders: 100만 행
lineitem: 600만 행

pgvector의 추정 (틀림):

$l.product_embedding \leftrightarrow query_vec < 0.5$ 선택도 = 33.3%
→ lineitem 필터링 후: 600만 × 33.3% = 200만 행

실제 선택도 (맞음):

< 0.5 선택도 = 0.001%
→ lineitem 필터링 후: 600만 × 0.001% = 60행

pgvector의 계획 (비효율):

1. orders Seq Scan: 100만 행 읽기
2. 각 행마다 lineitem Nested Loop Join
→ 600만 행 × 100만 = 6조 행 비교
3. 결과: 200만 행

최적 계획:

1. lineitem 벡터 필터: 60행
2. orders 조인: 60행 × 1회 조인
3. 결과: 60행

성능 차이:

pgvector: 6조 행 비교 필요 → 850초
최적: 60행 조인 → 2초
비율: 425배 느림

왜 부분적 근본 한계인가?

근본적 어려움:

- 벡터의 거리 분포는 쿼리에 따라 달라짐
- 고차원 벡터의 분포를 통계로 근사하기 어려움
- 사전에 통계를 수집하기 불가능 (무한히 많은 쿼리)

하지만 개선 가능:

- 쿼리 실행 중 정확한 선택도 학습 (Exqutor의 접근)
- 벡터 통계 수집 (새로운 자료구조)
- 인덱스 탐색으로 선택도 계산 (ECQ0의 접근)

개선 가능성: 높음. Exqutor가 증명한 대로 가능

1.4 근본 vs 구현: 분류 기준

논문의 핵심 기여는 각 한계를 **근본적 vs 구현 가능**으로 분류한 것이다:

근본적 한계 (아키텍처 수준): - 버퍼 풀 오버헤드: PostgreSQL의 동시성 제어 핵심 - 튜플 접근 오버헤드: MVCC 메타데이터 필수 - 공간 증폭: 8KB 페이지 기반 저장 구조

이들은 PostgreSQL을 PostgreSQL로 유지하면서 완전히 제거 불가능하다.

구현 개선 가능한 한계: - 인덱스 구축 시간: WAL 우회 (인덱스는 재생성 가능) + 병렬 구축 - 고정 선택도: 적응적 샘플링 또는 벡터 통계 수집

이들은 현재 구현의 문제이며, 소프트웨어 최적화로 해결 가능하다.

1.5 논문의 실험 방법론

신뢰성을 높이기 위한 방법:

1. 소스 코드 분석
2. pgvector/PASE C 코드 직접 검토
3. 각 함수 호출 추적
4. 마이크로벤치마크로 개별 컴포넌트 측정
5. 격리된 테스트 **각 오버헤드를 순차적 제거: 기본: 150ms 버퍼 풀 우회: 140ms (→ 10ms 절감) MVCC 우회: 120ms (→ 20ms 절감) WAL 비활성: 100ms (→ 20ms 절감) 메모리 직접: 8ms (→ 92ms 절감)**
6. 카운터 기반 분석
7. PostgreSQL 내부 통계 활용
8. 페이지 접근 횟수, 래치 경합 횟수 정량화

사용 데이터셋: - SIFT-128M: 1.28억 개 벡터 (128D) - GloVe: 119만 개 단어 임베딩 (300D) - MS COCO: 123만 이미지 특징 (2048D)

2. 핵심 개념 해설 (미팅용)

2.1 버퍼 풀(Buffer Pool)이란?

목적: 디스크와 메모리 사이의 속도 차이 극복

메모리 계층 접근 시간:

L1 캐시:	0.5ns	(64KB)
L2 캐시:	7ns	(256KB)
L3 캐시:	40ns	(8MB)
메인 메모리:	100ns	(16GB)
SSD:	100,000ns (1TB)	← 메모리의 1000배 느림
HDD:	10,000,000ns (10TB)	← 메모리의 100,000배 느림

해결책: 자주 사용하는 디스크 데이터를 메모리에 캐시

PostgreSQL 버퍼 풀:

설정: `shared_buffers = 4GB`
→ PostgreSQL이 메모리에서 관리하는 영역

운영:

1. 디스크에서 페이지 읽기 (8KB)
2. 메모리 버퍼 풀에 저장
3. 다음 요청 때 메모리에서 직접 읽음 (캐시 히트)
4. 필요하면 메모리 내용을 디스크에 다시 쓰기

효과:

캐시 히트율 95%면: 평균 접근 시간 = $100\text{ns} \times 0.05 + 100,000\text{ns} \times 0.95 \approx 95,000\text{ns}$
캐시 없음: 항상 100,000ns
→ 약 1.05배 개선 (큰 효과!)

왜 벡터 검색에 문제인가?

벡터 검색의 접근 패턴: - 무작위 접근(Random Access): 페이지들을 순차적이 아닌 무작위 순서로 접근 - 높은 빈도: 초당 수만~수십만 번 페이지 접근 - 결과: 캐시 미스율 높음 (50% 이상)

일반적인 SQL 쿼리 (순차 스캔):

페이지 0, 1, 2, 3, 4, ... 순차 접근
캐시 히트율: 99% (페이지 1,2,3,4는 버퍼에 이미 있음)

벡터 검색 (HNSW):

페이지 500, 10000, 25, 50000, 1, ... (무작위)
캐시 히트율: 30~50% (대부분 캐시 미스)

2.2 래치(Latch)란? Spin Lock과의 관계

문제: 여러 프로세스가 동일한 페이지에 접근하면?

프로세스 A: 페이지 100의 데이터 수정 중...
프로세스 B: 페이지 100을 동시에 읽으려고 함
프로세스 C: 페이지 100에 데이터를 추가하려고 함

→ 데이터 일관성 깨짐!

해결책: 래치(Latch)

래치는 매우 단기간의 락:

프로세스 A:

1. 래치 획득 (다른 모든 프로세스 진입 차단)
2. 데이터 수정
3. 래치 해제 (진입 재개)

프로세스 B:

1. 래치 획득 대기... (A가 놓을 때까지 대기)
2. A가 래치 해제
3. 래치 획득
4. 데이터 읽기
5. 래치 해제

래치의 특징:

- 매우 짧은 시간만 유지 (마이크로초 단위)
- 데드락이 없도록 설계 (항상 같은 순서로 획득)
- 트랜잭션 처리 중에는 유지 안 함 (짧은 작업용)

Spin Lock과의 관계:

래치 구현 방식: Spin Lock

래치 획득 시도:

```
while (래치가 busy) {  
    원자적 연산으로 래치 상태 확인  
    // CPU가 계속 폴링 (spinning)  
}  
래치 상태 = busy  
진입
```

문제점:

- busy-wait: CPU 사이클 낭비
- 많은 프로세스가 경쟁하면: 레지스터/캐시 오염
- 컨텍스트 스위칭 없음 (sleep 없음)

해결책:

- 경합이 높으면 sleep으로 전환 (futex)
- 적응형 spin lock: 상황에 따라 스핀 횟수 조정

벡터 검색에서의 영향:

HNSW 벡터 검색:

초당 100,000회 래치 획득 시도

경합 없을 때:

래치 획득: 50ns
데이터 접근: 100ns
래치 해제: 50ns
총: 200ns/페이지

경합 있을 때:

래치 획득 시도: 50ns
스핀 대기: 500ns (다른 프로세스가 래치 보유)
래치 획득: 50ns
데이터 접근: 100ns
래치 해제: 50ns
총: 750ns/페이지

비율: $750/200 = 3.75$ 배

100만 페이지 접근: $100만 \times 0.55ns = 550ms$ 오버헤드

2.3 MVCC (Multi-Version Concurrency Control)

문제: 트랜잭션의 일관성 유지

트랜잭션 A (TXN 100):

SELECT age FROM users WHERE id = 1

시간 T0: age = 30 읽음

트랜잭션 B (TXN 101): [TXN 100과 동시 실행]

UPDATE users SET age = 31 WHERE id = 1

시간 T1: age = 31로 변경

트랜잭션 A:

시간 T2: SELECT age FROM users WHERE id = 1
age = ? (30인가 31인가?)

해결책: MVCC

각 행의 여러 버전을 유지한다:

사용자 ID=1:

버전 1: age=30, xmin=50, xmax=100

버전 2: age=31, xmin=101, xmax= ∞

트랜잭션 A (스냅샷 100):

TXN 100이 시작할 때 100 이상의 TXN은 "보이지 않음"

버전 1 확인: $xmin(50) < 100$ 이고 $xmax(100) \leq 100$

→ 이 버전은 TXN 100 시작 이전에 삭제됨

→ 보이지 않음

버전 2 확인: $xmin(101) > 100$

→ 이 버전은 TXN 100 시작 이후에 생성됨

→ 보이지 않음

결론: TXN 100의 시점에 age=30

트랜잭션 B:

스냅샷 101: 모든 TXN < 101 완료된 것으로 봄

버전 2 확인: $xmin(101) = 101$, $xmax(\infty)$

→ 현재 버전

→ age=31 보임

왜 벡터 검색에 오버헤드인가?

벡터 1개 접근:

1. 데이터 읽기 전: MVCC 가시성 확인 필수
 - 현재 트랜잭션의 스냅샷 읽기
 - 튜플의 xmin/xmax 확인
 - 메모리 참조 5~10회
 - 약 100ns 추가
2. 벡터 100만 개 검색:
 - 100만 개 × 100ns = 100ms 추가 오버헤드
3. 비교:
 - 거리 계산: 100ms
 - MVCC 확인: 100ms
 - 비율: 50% 오버헤드

벡터 DB에서는 불필요한 이유:

벡터 데이터의 특성:

- 대부분 삽입 후 변경 없음
- 삭제도 거의 없음
- 동시에 쓰는 경우가 드물

따라서:

- 모든 벡터는 "최신 버전"
- MVCC 확인 불필요
- 버전 간 선택 불필요
- xmin/xmax 메타데이터 불필요

2.4 WAL (Write-Ahead Log)

목적: 장애 발생 시 데이터 복구

장애 시나리오:

정상 상황:

데이터 A 읽기 → 수정 → 메모리에 쓰기 → 나중에 디스크에 쓰기

서버 크래시:

메모리의 수정 사항이 모두 날아감!
원본 디스크 데이터만 남음

→ 수정 내용이 완전히 손실됨

해결책: WAL

모든 변경을 먼저 로그에 기록:

1. 데이터 A를 메모리에서 수정 시작
2. WAL에 "데이터 A 수정" 로그 기록 (디스크에 동기 쓰기)
3. 로그 기록 확인
4. 실제 메모리 데이터 수정
5. 나중에 디스크에 쓰기

서버 크래시 발생:

메모리 데이터 손실

하지만 WAL 로그는 디스크에 안전하게 저장

→ 복구: WAL 로그 읽기 → 메모리 상태 재구성

핵심 원칙:

"로그 먼저 (Write-Ahead), 데이터 나중"

성능 비용:

벡터 삽입 시:

1. 메모리에 벡터 저장
2. WAL에 "벡터 삽입" 기록
 - └ fsync() 호출: 디스크 동기 쓰기
 - └ 대기: 1~10ms (디스크 속도에 따라)
3. 확인 메시지 반환
4. 다음 벡터 삽입 (2번 반복)

효과:

디스크 I/O가 벡터 삽입의 병목

$1000만\ 벡터 \times 5ms = 50,000초 = 13.8시간$

인덱스 구축 시에만 피할 수 있는 이유:

인덱스 재구성:

기본 구축 방식 (안전):

1. 벡터 삽입 (WAL 로그)
 2. HNSW 인덱스 갱신 (WAL 로그)
- 매우 느림

대안 (재구성):

1. 벡터 일단 일반 테이블에 삽입 (체크포인트 때 일괄 쓰기)
2. 완료 후 HNSW 인덱스 빌드
3. 새 인덱스로 교체

→ 더 빠름 (일괄 처리 가능)

실패 시:

- 기존 인덱스 유지
- 새 인덱스 폐기 (재생성 가능이므로 안전)

2.5 공간 증폭의 구조적 원인

8KB 페이지 단위 저장의 문제:

HNSW 그래프 저장:

벡터 1개 = 약 6KB ($1536D \times 4B$)

간선 정보 = 약 200B (이웃 50개 $\times$ 4B 포인터)

합계: 약 6.2KB

PostgreSQL 페이지 (8KB):

페이지 헤더: 24B	
튜플 헤더: 23B	
데이터: 6.2KB	
패딩: 1.7KB (낭비!)	

→ 낭비율: $1.7KB / 8KB = 21\%$

더 심한 경우:

만약 간선 정보만 저장하면 (200B):

페이지 헤더: 24B	
튜플 헤더: 23B	
데이터: 200B	
패딩: 7.7KB (낭비!)	

→ 낭비율: $7.7KB / 8KB = 96\%$

누적 효과:

1000만 벡터 HNSW 인덱스:

Faiss:

벡터 정보: 60GB (기본)

HNSW 간선: 30GB (이웃 50개 $\times$ 포인터)

총: 90GB

메모리에 올라감

PostgreSQL:

페이지 오버헤드: $90GB / 0.8 = 112.5GB$ (20% 오버헤드)

MVCC 메타데이터: 240MB

정렬 임시 공간: 100GB (재정렬)

총: 약 200GB

메모리 부족 → 일부 SSD로 스푼

성능 영향:

Faiss: 메모리 100ns 접근

PostgreSQL: SSD 100,000ns 접근 (1000배 느림)

2.6 선택도 추정의 구조적 한계

왜 벡터 거리는 통계로 추정 불가능한가?

스칼라 데이터 (통계 가능):

나이 분포: [10, 15, 20, 25, 30, 35, 40, 45, 50, ...]

쿼리: WHERE age > 30

추정: 히스토그램 사용

- 30 이상의 데이터 비율 계산
- 정확도 90% 이상

선택도는 데이터 분포에만 의존

벡터 데이터 (통계 불가능):

벡터 분포: 1536차원의 구체(sphere) 표면에 분산

쿼리: WHERE embedding <-> query_vec < 0.5

같은 데이터셋에서도:

- 쿼리 1 (query_vec_a): 100개 히트 (선택도 0.001%)
- 쿼리 2 (query_vec_b): 1000개 히트 (선택도 0.01%)
- 쿼리 3 (query_vec_c): 100개 히트 (선택도 0.001%)

→ 쿼리에 따라 완전히 다름

원인:

- 고차원 벡터 공간에서 거리의 분포는 쿼리 벡터에 의존
- 사전에 통계 수집 불가능 (쿼리마다 다름)
- 실행 시간에도 효율적 추정 불가능 (계산 비용 높음)

2.7 Faiss vs PostgreSQL 아키텍처 비교

Faiss:

메모리 기반:

- 전체 인덱스가 메모리에 로드
- 버퍼 풀 없음 (직접 접근)
- MVCC 없음 (불변 인덱스)
- WAL 없음 (장애 복구 기능 없음)
- 8KB 페이지 없음 (가변 크기 저장)
- 고정 선택도 없음 (검색만 수행)

결과: 벡터 검색에 최적화

제한사항:

- 디스크 지원 없음 (인메모리만)
- ACID 미지원
- SQL 미지원
- 동시성 제어 미지원

PostgreSQL:

디스크 기반:

- 버퍼 풀 사용 (메모리-디스크 일관성)
- MVCC (동시 읽기/쓰기)
- WAL (장애 복구)
- 8KB 페이지 (동시성 제어)
- 통계 기반 최적화 (일반 테이블)

결과: 범용 데이터 관리에 최적화

장점:

- SQL 지원
- ACID 지원
- 동시 사용자 지원
- 디스크 영속성

제한사항:

- 벡터 검색에 오버헤드

→ pgvector = 범용성 vs 성능 트레이드오프

2.8 "근본적 한계"와 "구현 개선 가능" 한계의 구분 기준

근본적 한계의 특징:

1. 아키텍처에 내재적
2. 시스템의 기본 설계 원칙에서 비롯됨

3. 예: 버퍼 풀 (PostgreSQL의 동시성 기반)

4. 제거하면 다른 기능이 깨짐

5. 버퍼 풀 제거 → MVCC 불가능 → 동시성 제어 불가

6. MVCC 제거 → 트랜잭션 일관성 깨짐

7. 범용성과 성능의 트레이드오프

8. 범용성을 유지하려면 이 기능 필수

9. 성능만 원하면 기능 제거 가능

구현 개선 가능한 한계의 특징:

1. 설계 자체는 좋지만 구현이 미흡

2. 예: 인덱스 구축 시 WAL 로깅

3. WAL은 필요하지만, 인덱스 재생성으로 우회 가능

4. 다른 기능을 해치지 않고도 개선 가능

5. 예: 선택도 추정

6. 통계 개선 → 동시성/일관성 영향 없음

7. 최적화 기술로 해결 가능

8. 예: 적응적 샘플링 (Exqutor)

9. 쿼리 실행 중 정확한 정보 학습

3. Exqutor와의 연결고리

3.1 5가지 한계 중 Exqutor의 역할

한계 5: 고정 선택도(Fixed Selectivity) — Exqutor가 직접 해결

문제:

- pgvector의 모든 벡터 필터에서 선택도 = 33.3% 가정
- 쿼리 계획 최적화 불가능
- 비효율적 조인 순서 선택
- 성능 저하 (425배까지)

Exqutor의 해결책:

ECQO (Estimated Cardinality Query Optimization):

1. 쿼리 실행 중 벡터 필터의 정확한 선택도 학습
2. 중간 결과의 카디널리티 추정
3. 이후 조인 순서 동적 재최적화

결과:

- pgvector의 33.3% → 정확한 값 (예: 0.1%)
- 조인 순서 자동 최적화
- 성능 향상 425배까지 가능

한계 1~4: 근본적 한계 — Exqutor가 직접 해결하지 않음

한계 1 (버퍼 풀 래치): 여전히 존재

- 각 페이지 접근 시 래치 필수
- 약 20배 오버헤드
- 하드웨어/아키텍처 수준의 개선 필요

한계 2 (튜플 MVCC): 여전히 존재

- 각 벡터의 xmin/xmax 확인 필수
- 약 30% 오버헤드

한계 3 (공간 증폭): 여전히 존재

- 8KB 페이지 기반 저장
- 약 8배 오버헤드

한계 4 (인덱스 구축): 여전히 존재

- WAL 로깅 필수 (트랜잭션 안전성)
- 약 96배 느림

3.2 상대적 영향 감소 분석

Exqutor 적용 전 (pgvector 기본):

성능 저하의 주요 원인들:

쿼리 계획 최적화 불가능 (한계 5):

→ 425배 느림 (가장 큼)

버퍼 풀 래치 경합 (한계 1):

→ 20배 느림

튜플 MVCC 확인 (한계 2):

→ 1.3배 느림

공간 증폭으로 인한 I/O (한계 3):

→ 부분적으로 한계 1에 포함됨

인덱스 구축 (한계 4):

→ 별도 이슈 (건물 기간에만 영향)

종합 효과 (곱셈):

$425배 \times 20배 \times 1.3배 = 11,050배$ 느림

실제 측정:

Faiss: 100ms

pgvector: 약 1,100초 (11,000배)

Exqutor 적용 후:

한계 5 해결: 425배 → 1배 (완전 해결)

남은 오버헤드:

- 버퍼 풀 래치 (한계 1): 20배
- 튜플 MVCC (한계 2): 1.3배
- 공간 증폭 (한계 3): 부분 포함
- 인덱스 구축 (한계 4): N/A

중합 효과:

$1\text{배} \times 20\text{배} \times 1.3\text{배} = 26\text{배}$

결과:

pgvector + Exqutor: 약 2,600ms

하지만 이론값: $100\text{ms} \times 26 = 2,600\text{ms}$

실제 성능:

Faiss: 100ms

pgvector: 1,100초

pgvector + Exqutor: 2~5초

비율:

$1,100\text{초} / 2.5\text{초} = 440\text{배}$ 향상!

또는 측정: $1,000\text{배} \rightarrow 20\text{배} = 50\text{배}$ 향상

3.3 구체적 사례: 조인 쿼리

예제 쿼리:

```
SELECT
  o.order_id,
  o.customer_id,
  COUNT(*) as item_count
FROM orders o
JOIN lineitem l ON o.id = l.order_id
WHERE l.product_embedding <-> query_vec < 0.5
GROUP BY o.id, o.customer_id
```

테이블 크기: - orders: 100만 행 - lineitem: 600만 행

pgvector (기본):

선택도 추정 (틀림): 33.3%

WHERE 절 선택도 = 33.3%

→ lineitem 필터 후: 600만 × 0.333 = 200만 행

계획:

1. orders Seq Scan (100만 행)
2. Nested Loop Join
 - orders의 각 행마다 lineitem 스캔 (600만 행)
 - 총 600만 × 100만 = 6조 행 비교
3. WHERE 절 필터: 6조 × 0.333 = 2조 행 필터링
4. GROUP BY: 2조 행 그룹화

성능:

- 6조 행 비교: 매우 느림
- 실행 시간: 약 850초

pgvector + Exqutor:

쿼리 실행 중 선택도 학습:

1단계: 초기 계획 (pgvector 추정)

- 200만 행 예상

2단계: 쿼리 실행 중 정확한 선택도 발견

- 실제 WHERE 절 히트: 100행 (0.0016%)
- 예상: 200만 행
- 오류: 20,000배!

3단계: 계획 재최적화

- lineitem 먼저 벡터 필터 (100행)
- 그 다음 orders 조인 (100행)

수정된 계획:

1. lineitem 벡터 필터 (WHERE 절) → 100행
2. orders 조인 → 100행 결과

성능:

- 100행만 처리
- 실행 시간: 약 2초

비율: 850초 / 2초 = 425배 향상!

3.4 이 논문이 Exqutor를 정당화하는 방식

논문의 발견:

1. pgvector는 느리다 (실제 측정)

2. Faiss 대비 100~1000배 느림
3. **원인 분석** (5가지 한계)
4. 한계 5: 선택도 추정 (425배)
5. 한계 1: 버퍼 풀 (20배)
6. 기타: 나머지
7. **일부는 구현 개선 가능** (아니 근본 한계 아님)
8. 한계 5: 통계/샘플링 개선 가능
9. 한계 4: WAL 우회 가능

이것이 Exqutor를 정당화하는 이유:

논문: "선택도 추정은 구현 개선으로 해결 가능"
→ 425배 성능 향상 가능성 있음

Exqutor: "적응적 샘플링으로 선택도를 정확히 추정"
→ 논문의 제안을 실제 구현
→ 425배 성능 향상 달성 (부분적으로)

결론:

이 논문이 없었다면:

- pgvector 성능 저하의 원인이 불명확
- Exqutor의 필요성이 입증되지 않음

이 논문과 함께:

- 성능 저하의 구체적 원인 명시
- 개선 가능한 영역 식별
- Exqutor의 기여도 명확화

4. 예상 질문과 답변 (Q&A)

Q1: 이 논문이 뭘 밝혀냈어요?

A: 세 가지 핵심 발견:

1. pgvector는 왜 느린가?
2. Faiss 대비 100~1,000배 느림 (실제 측정)
3. **원인이 뭔가?**

4. 5가지 구체적 원인 식별:

- 버퍼 풀 래치 (20배)
- MVCC 오버헤드 (30%)
- 공간 증폭 (8배)
- 인덱스 구축 (96배)
- 고정 선택도 (425배)

5. 모두 해결 불가능한가?

6. 아니다. 일부는 근본 한계(버퍼 풀, MVCC)

7. 일부는 구현 개선 가능(선택도, 인덱스 구축)

Q2: "근본적 한계"가 뭐예요? "구현 한계"랑 뭐가 달라요?

A: 제거 가능성의 차이:

근본적 한계 (제거 불가): - 시스템의 기본 설계 원칙에서 비롯됨 - 제거하면 다른 기능이 깨짐 - 예: 버퍼 풀 제거 → MVCC 불가능 → 동시성 제어 깨짐

구현 한계 (개선 가능): - 현재 구현 방식의 선택일 뿐 - 다른 방식으로도 같은 기능 구현 가능 - 예: 인덱스 구축 시 WAL 로깅 - 현재: 벡터마다 WAL 기록 (느림) - 대안: 구축 완료 후 검증 (빠름) - 둘 다 안전성 달성 가능

Q3: 버퍼 풀이 뭐예요? 왜 벡터 검색에 문제예요?

A: 두 부분으로 설명:

버퍼 풀이란?

메모리와 디스크의 속도 차이 극복 장치:

메모리: 100ns 접근 시간

SSD: 100,000ns 접근 시간 (1000배 느림)

솔루션: 자주 사용하는 디스크 데이터를 메모리에 캐시

왜 벡터에 문제인가?

일반 SQL:

```
SELECT * FROM users WHERE age > 30
```

접근 패턴: 순차적

페이지 0 → 1 → 2 → 3 → ...

캐시 히트율: 99% (이미 로드된 페이지)

결과: 효율적

벡터 검색:

HNSW 노드 탐색

접근 패턴: 무작위

페이지 500 → 10000 → 25 → 50000 → 1 → ...

캐시 히트율: 30% (거의 캐시 미스)

결과: 항상 메모리 부족 상태

→ 메모리에서 못 찾음 → 디스크 접근 필요

→ 100ns vs 100,000ns = 1000배 차이

Q4: 래치(Latch)가 뭐예요?

A: 매우 짧은 시간의 락:

목적:

여러 프로세스가 동일 페이지 접근 시 데이터 일관성 유지

프로세스 A: 페이지 수정 중 (1마이크로초)

프로세스 B: 동시에 페이지 읽기 시도

→ 래치로 대기 (A의 수정 끝날 때까지)

특징: - 매우 짧은 보유 시간 (마이크로초) - Spin lock으로 구현 (CPU가 계속 폴링) - 데드락 방지 설계 (항상 같은 순서로 획득)

벡터 검색에서의 오버헤드:

HNSW 탐색: 초당 100,000회 래치 획득

경합이 없을 때: 200ns/페이지

경합이 있을 때: 750ns/페이지 (3.75배)

백만 페이지 접근:

추가 오버헤드: 550ms

전체 검색 시간 150ms 대비 매우 큼

Q5: MVCC가 뭐예요? 벡터에 왜 불필요해요?

A: MVCC = 동시 읽기/쓰기를 가능하게 하는 기법

예:

일반 락 방식 (MVCC 없음):

TXN A (읽기): 데이터 읽으려고 락 요청

TXN B (쓰기): 데이터 쓰고 있음 (락 보유)

TXN A: 대기... (TXN B 완료까지 대기)

MVCC (다중 버전 관리):

TXN A (읽기): "과거 버전" 읽기 (락 불필요)

TXN B (쓰기): "새 버전" 생성 (동시 진행)

→ 동시 읽기/쓰기 가능

구현 비용:

각 데이터마다 메타데이터:

xmin: 이 버전을 생성한 트랜잭션 ID (4B)

xmax: 이 버전을 삭제한 트랜잭션 ID (4B)

cmin/cmax: 명령 ID (2B)

총 10B 메타데이터

접근할 때마다:

1. 메타데이터 읽기

2. 현재 트랜잭션의 스냅샷과 비교

3. "이 데이터가 보이는가?" 판단

→ 추가 CPU/메모리 사용

벡터에 불필요한 이유:

벡터 데이터의 특성:

- 보통 읽기 전용
- 삽입 후 수정 거의 없음
- 동시 쓰기 없음

결론:

- MVCC 기능 불필요
- 메타데이터 낭비
- 접근 시마다 체크 오버헤드

Q6: 공간 증폭이 8배나 된다고요?

A: 네, 여러 원인이 있습니다:

원인 1: 8KB 페이지 단위 저장의 낭비

HNSW 저장:

벡터: 6KB
간선: 200B
총: 6.2KB

8KB 페이지에 저장:

└ 페이지 헤더: 24B
└ 튜플 헤더: 23B
└ 데이터: 6.2KB
└ 패딩: 1.7KB (낭비)

낭비율: 21%

간선만 저장하면:

└ 패딩: 7.7KB (낭비!)
낭비율: 96%

원인 2: MVCC 메타데이터

1000만 벡터 × 24B = 240MB 추가

원인 3: 정렬 임시 공간

인덱스 구축 시 정렬:

중간 정렬 결과를 디스크에 임시 저장
약 100GB (구축 완료 후 폐기)

종합 결과:

Faiss (메모리): 90GB

PostgreSQL: 약 200GB

비율: $200/90 \approx 2.2$ 배

하지만 메모리 적재 시:

Faiss: 메모리에 모두 올라감

PostgreSQL: 메모리 부족 → SSD로 스푼

실질적 접근 시간:

- Faiss: 100ns

- PostgreSQL: 100,000ns (SSD)

- 성능 차이: 1000배!

Q7: 인덱스 구축이 96배 느리다고요?

A: 맞습니다. WAL 로깅 때문입니다:

원인:

벡터 1개를 인덱스에 추가할 때:

1단계: 메모리에 저장

2단계: WAL 로그에 기록 (디스크 동기 I/O)

→ 1~10ms 대기 (매우 느림)

3단계: 확인 응답

→ 벡터 1개 = 5~10ms

1000만 벡터:

$10ms \times 1000만 = 100,000,000ms = 11,111시간$

실제 측정:

48시간 = 약 96배 느림 (벡터당 약 20ms)

비교:

Faiss (메모리): 30분 (약 0.0018ms/벡터)

해결 방법 (논문에서 제시):

옵션 1: 인덱스 구축 중 WAL 우회

- 인덱스는 재생성 가능하므로 로그 불필요
- 벡터 삽입에 WAL 적용하되, 인덱스 갱신은 우회
- 약 50배 개선 가능

옵션 2: 병렬 구축

- 여러 프로세스가 다른 파티션 동시 구축
- 약 4배 개선 (4 프로세스 사용 시)

옵션 3: 배치 처리

- 벡터를 일괄 삽입
- WAL 로그 일괄 쓰기
- I/O 횟수 감소
- 약 10배 개선 가능

Q8: 고정 선택도가 33.3%라는데 이게 왜 치명적이에요?

A: 쿼리 계획의 극단적 비효율을 초래합니다:

예시:

쿼리:

```
SELECT * FROM lineitem
WHERE product_embedding <-> query_vec < 0.5
```

실제 결과: 100행

pgvector 추정: 600만 행 × 33.3% = 200만 행

오류: 20,000배 과대 추정!

조인 쿼리에서의 영향:

```
SELECT o.order_id
FROM orders o (100만 행)
JOIN lineitem l (600만 행)
  ON o.id = l.order_id
WHERE l.embedding <-> query_vec < 0.5
```

pgvector의 선택도 추정: 33.3%
→ lineitem에서 200만 행 예상

계획 1 (pgvector 추정 기반):
orders Seq Scan (100만 행)
└ Nested Loop Join
 └ lineitem 스캔 (600만 행)

총 비교: 600만 × 100만 = 6조 행
WHERE 절 필터: 6조 × 0.333 = 2조 행 처리
성능: 850초

계획 2 (정확한 선택도):
lineitem 벡터 필터 (WHERE) → 100행
orders 조인 → 100행

성능: 2초

비율: 850 / 2 = 425배 차이!

왜 피하기 어려운가?

벡터 거리는 쿼리에 따라 분포가 완전히 다름:

쿼리1: 거리 < 0.5 → 100행 히트
쿼리2: 거리 < 0.5 → 10,000행 히트

같은 데이터인데도 결과가 다름

따라서 사전 통계 불가능
실행 시 동적 추정 필요 (Exqutor의 접근)

Q9: 그럼 pgvector 쓰면 안 되는 건가요?

A: 상황에 따라 다릅니다:

pgvector를 써도 괜찮은 경우:

1. 벡터 검색이 주요 작업이 아닐 때
 - 일반 SQL 쿼리 + 가끔 벡터 검색
 - 전체 성능의 5% 이하
2. 데이터 크기가 작을 때
 - < 100만 벡터
 - Faiss (메모리 전체 로드)보다 관리 편함
 - 성능 저하가 허용 범위 내
3. SQL 통합이 중요할 때
 - 벡터 + 메타데이터 결합 검색
 - 조인/필터 복잡함
 - PostgreSQL이 선택지 없음
4. ACID 필요할 때
 - 읽기/쓰기 일관성 필수
 - Faiss (읽기 전용) 사용 불가

pgvector를 피해야 하는 경우:

1. 벡터 검색이 주요 작업일 때
 - 실시간 검색 API
 - 빠른 응답 시간 필수
 - 1초 이상 대기 불가능
2. 매우 큰 데이터 (수천만 벡터)
 - 메모리 부족 → SSD 스필
 - 성능 급격히 저하
3. 높은 처리량 필요할 때
 - 초당 수천 쿼리
 - 래치 경쟁 심각
4. 성능이 사업에 직결될 때
 - 검색 성능 = 사용자 경험
 - 100ms vs 1000ms의 차이가 큼

결론:

pgvector는:

- 범용성 + 약간의 벡터 기능 필요
- 성능보다 SQL 통합이 우선

이 경우에 선택할 것.

벡터 성능이 중요하다면:

- Faiss/Milvus (전문 DB)
- pgvector + Exqutor (하이브리드)

Q10: Faiss가 더 좋으면 Faiss 쓰면 되지 않나요?

A: Faiss의 제한이 크기 때문입니다:

Faiss의 문제점:

1. 인메모리만 지원
 - 데이터가 메모리에 모두 들어가야 함
 - 1000만 벡터 = ~90GB 메모리 필요
 - 서버 메모리가 512GB라도, 대규모는 불가능
2. 읽기 전용
 - 데이터 추가/수정 불가능
 - 벡터 데이터가 고정되어야 함
 - 실시간 업데이트 필요하면 전체 재구축
3. SQL 미지원
 - "embedding < 0.5 AND created_at > '2024-01-01'"
 - 이런 복합 쿼리 불가능
 - 애플리케이션에서 별도 필터링 필요
4. ACID 미지원
 - 트랜잭션 없음
 - 일관성 보장 없음
 - 금융 데이터 등에 사용 불가
5. 단일 머신만 지원
 - 분산 처리 불가능
 - 대규모 시스템에서 확장 어려움

pgvector의 장점:

1. 디스크 지원
 - 데이터 크기 제약 없음
 - SSD/HDD에 저장 가능
2. 동적 업데이트
 - 실시간 벡터 추가/수정 가능
 - 뮤지컬 임베딩 같이 자주 갱신되는 데이터
3. SQL 통합
 - 복합 쿼리 가능
 - WHERE/JOIN/GROUP BY 모두 사용
4. ACID 지원
 - 트랜잭션 안전성
 - 금융/의료 데이터에 안전
5. 분산 지원 (Citus 등)
 - PostgreSQL을 여러 서버로 확장 가능

결론:

Faiss vs pgvector:

- Faiss는 "벡터 특화, 제한 많음"
- pgvector는 "범용성, 약간 느림"

선택:

벡터만 필요하고 메모리 충분: Faiss
벡터 + SQL 통합 필요: pgvector
벡터 성능 + SQL 통합: pgvector + Exqutor

Q11: Exqutor는 이 5가지 중 뭘 해결해요?

A: 한계 5 (고정 선택도)를 직접 해결합니다:

한계 5 해결:

방법: ECQO (Estimated Cardinality Query Optimization)

동작:

1. 쿼리 실행 중 벡터 필터의 정확한 선택도 학습
2. 중간 결과의 카디널리티 정확히 파악
3. 이후 조인 순서 동적 재최적화
4. 비효율적 계획 → 효율적 계획으로 변경

결과:

- 선택도 추정 오류: 33,300배 → 0배 (정확)
- 쿼리 성능: 425배 향상

다른 한계들:

한계 1 (버퍼 풀 래치): 미해결

- 근본 한계라 Exqutor로 불가능
- 약 20배 오버헤드 여전

한계 2 (MVCC): 미해결

- 근본 한계
- 약 30% 오버헤드 여전

한계 3 (공간 증폭): 미해결

- 근본 한계
- 약 8배 오버헤드 여전

한계 4 (인덱스 구축): 미해결

- 별도 최적화 필요 (Exqutor의 범위 밖)
- 약 96배 느림 여전

Q12: 나머지 한계는 어떻게 해요?

A: 서로 다른 접근이 필요합니다:

한계 1 (버퍼 풀 래치):

근본 한계: 완전히 제거 불가능

해결책:

- VBASE: 벡터 전용 메모리 영역
(일반 버퍼 풀 우회)
- 아키텍처 수준의 변경 필요
- PostgreSQL 핵심 개발자 협력 필수

기대 효과:

- 20배 오버헤드 → 1배 정도로 감소

한계 2 (MVCC):

근본 한계: 완전히 제거 불가능

해결책:

- 벡터 데이터를 immutable로 취급
- MVCC 체크 단순화 (변경 불가능이므로 버전 선택 불필요)
- 메타데이터 축소 가능

기대 효과:

- 30% 오버헤드 → 5% 정도로 감소

한계 3 (공간 증폭):

근본 한계: 8KB 페이지 기반 저장 구조 근본 문제

해결책:

- 벡터 전용 저장소 설계
- 가변 크기 저장 (동시성 관리 복잡도 증가)
- 또는 페이지 크기 조정 (PostgreSQL 버전별로 다름)

기대 효과:

- 8배 오버헤드 → 2배 정도로 감소

한계 4 (인덱스 구축):

부분 해결 가능:

방법 1: WAL 우회 (인덱스 구축 시)

- 인덱스는 재생성 가능하므로 로그 불필요
- 약 50배 개선

방법 2: 병렬 구축

- 여러 프로세스 동시 작업
- 약 4배 개선 (4 프로세스)

방법 3: 배치 처리

- 벡터 일괄 삽입
- 약 10배 개선

종합: 96배 → 약 5배 정도로 감소 가능

Exqutor의 역할:

Exqutor는 "한계 5만" 집중 해결

- 425배 성능 향상 (가장 큼)
- 다른 한계는 별도 해결 필요

하지만 장점:

- 구현 비용 낮음 (쿼리 최적화만)
- 즉시 적용 가능
- 다른 개선과 조합 가능

결과:

pgvector (기본): 1,000배 느림
pgvector + Exqutor: 20배 느림
pgvector + Exqutor + 다른 최적화: 5배 이하도 가능

5. 미팅 대본 (5분 분량)

안녕하세요. 저는 Exqutor 프로젝트의 [이름]입니다.

오늘 소개할 논문은 "**PostgreSQL에서 벡터 데이터 관리의 근본적 한계**" (ICDE 2024, Purdue University)입니다. 이 논문이 우리 프로젝트를 정당화하는 핵심 근거입니다.

배경: pgvector는 왜 느린가?

먼저 문제를 정의하겠습니다. PostgreSQL의 pgvector 확장은 Faiss라는 전문 벡터 라이브러리보다 **100배에서 1000배 느립니다**.

실제 성능:

- Faiss (메모리 기반): 100ms
- pgvector (디스크 기반): 1,100초
- 비율: 11,000배

문제는 "왜"입니다. 구현이 나쁜 건가요, 아니면 아키텍처 자체가 문제인가요? 이 논문이 정확히 이를 분석합니다.

발견: 5가지 근본 한계

논문은 체계적인 실험을 통해 **5가지 핵심 한계**를 식별했습니다. 각각을 간단히 설명하겠습니다.

첫 번째: 버퍼 풀 래치(Latch) 경합

PostgreSQL은 모든 데이터에 접근할 때 "래치"라는 짧은 락을 사용합니다. 이는 동시성 제어 때문에 필수입니다. 그런데 벡터 검색(HNSW)은 무작위로 수백만 개의 페이지를 접근합니다. 각 접근마다 래치 획득/해제의 오버헤드가 발생합니다.

측정 결과: 약 20배 성능 저하

두 번째: 튜플 접근 오버헤드

PostgreSQL의 모든 행은 xmin, xmax 같은 메타데이터를 가집니다. MVCC(동시 읽기/쓰기)를 지원하기 위함인데, 벡터는 보통 읽기 전용이라 불필요합니다.

측정 결과: 약 30% 성능 저하

세 번째: 공간 증폭

PostgreSQL은 데이터를 8KB 단위의 페이지로 관리합니다. HNSW 그래프는 이 페이지에 맞춰 저장되면서 큰 낭비가 생깁니다. 1000만 벡터의 인덱스가 Faiss의 90GB에서 PostgreSQL의 200GB로 팽창합니다.

측정 결과: 약 8배 공간 증가, 메모리 부족 시 SSD 접근으로 성능 1000배 저하

네 번째: 인덱스 구축 시간

WAL(Write-Ahead Log)이라는 안전장치 때문에 모든 변경이 먼저 로그에 기록됩니다. 이 때문에 1000만 벡터 인덱싱이 48시간(Faiss는 30분)이 됩니다.

측정 결과: 약 96배 느림

다섯 번째: 고정 선택도

이것이 가장 치명적입니다. PostgreSQL은 벡터 필터의 선택도(반환 행 수)를 "33.3%"로 고정 가정합니다. 하지만 실제로는 0.001% ~ 1%로 크게 다릅니다.

예시: 1000만 벡터에서 가까운 100개 찾기
- pgvector 추정: 333만 행 (33.3%)
- 실제: 100행 (0.001%)
- 오류: 33,300배!

이는 쿼리 계획을 극도로 비효율적으로 만듭니다. 100만 행 orders 테이블과 600만 행 lineitem을 조인할 때, pgvector는 6조행을 비교하게 됩니다(정확한 추정이면 100행).

측정 결과: 약 425배 성능 저하

분석: 근본 vs 구현

이 다섯 가지 한계를 분류하면:

근본적 한계 (아키텍처 수준): - 버퍼 풀 (동시성 제어 필수) - MVCC (트랜잭션 안전성 필수) - 공간 증폭 (8KB 페이지 구조)

이들은 PostgreSQL을 유지하면서 완전히 제거할 수 없습니다.

구현 개선 가능한 한계: - 인덱스 구축 (WAL 우회 가능) - 고정 선택도 (통계/샘플링 개선 가능)

이들은 현재 구현의 선택일 뿐, 다른 방식으로 같은 기능을 더 효율적으로 구현할 수 있습니다.

Exqutor의 역할

이것이 Exqutor가 나오는 이유입니다. 우리의 ECQO(Estimated Cardinality Query Optimization) 기법은 **다섯 번째 한계를 직접 해결**합니다.

동작: 1. 쿼리 실행 중 벡터 필터의 정확한 선택도를 학습합니다 2. 중간 결과의 카디널리티를 정확히 파악합니다 3. 이를 바탕으로 조인 순서를 동적으로 재최적화합니다

결과: - 선택도 추정 오류: 33,300배 → 0배 - 쿼리 성능: 425배 향상 - 전체: pgvector 1,100초 → 2~5초

실제로는 나머지 한계(버퍼 풀, MVCC 등)이 약 20배 여전히 있지만, 가장 심각한 문제(선택도)를 해결함으로써 전체적으로 **50배의 성능 향상**을 달성했습니다.

결론

이 논문의 기여: 1. pgvector의 문제를 과학적으로 분석 2. 일부는 근본, 일부는 개선 가능임을 명시 3. 개선 가능한 영역 식별

Exqutor의 기여: 1. 식별된 개선 가능 영역(한계 5)을 실제로 해결 2. 구현 비용이 적으면서 효과가 큼 3. 다른 최적화와 조합 가능

결과적으로, pgvector가 "느린 것"은 당연합니다. 범용성과 성능의 트레이드오프이기 때문입니다. 하지만 "구현 개선으로 성능을 대폭 향상시킬 수 있다"는 것을 이 논문이 명확히 하였고, Exqutor가 이를 증명했습니다.

감사합니다.

부록

A. 기술용어 정리

- **MVCC**: 여러 버전의 데이터를 동시에 관리하여 읽기/쓰기 충돌 해결
- **WAL**: Write-Ahead Log, 변경 전 로그 기록으로 장애 복구 보장
- **Latch**: 페이지 수준의 짧은 락, 동시 접근 제어
- **Selectivity**: 쿼리 필터가 전체 데이터 중 몇%를 반환하는가
- **Cardinality**: 테이블의 행 수 또는 중간 결과의 행 수
- **HNSW**: Hierarchical Navigable Small World, 계층적 그래프 기반 벡터 인덱스
- **ECQO**: Exqutor의 핵심 기법, 쿼리 실행 중 정확한 카디널리티 추정

B. 추가 분석 포인트

논문의 한계:

1. Faiss와의 비교는 공정한가?
2. Faiss는 메모리 전용, PostgreSQL은 디스크 지원
3. 기능이 완전히 다름 (비교 대상 부적절 가능)
4. 벤치마크 워크로드는 대표적인가?
5. 순수 벡터 검색만 테스트
6. 실제 응용은 더 복잡한 쿼리
7. pgvector의 최신 버전은?
8. 논문은 0.5.x 기준
9. 0.7.x 이상은 성능 개선되었을 가능성

Exqutor의 한계:

1. 모든 한계를 해결하지 못함
2. 버퍼 풀, MVCC, 공간 증폭은 여전함
3. 추가 최적화 필요
4. 선택도 추정의 오버헤드

5. 쿼리 실행 중 샘플링 비용
6. 작은 결과셋은 효율성 낮을 수 있음
7. 다른 데이터베이스 적용 가능성
8. MySQL, Oracle에서는?
9. 일반화 가능성 미확인